inputdata 구조 설명서

문제지 업로드 기능: 파일별 역할 · 연결 방식 · .env 사용 위치 | 기준: 2026-11-09 개발본 (23개 파일)

23	약 7,000	4	11	46
파일	줄	DB 표	API	시험 통과

Written for: 백엔드 담당자 — 서버에 반영하고 고칠 사람. 이 문서는 `.env` 파일을 열지 않고 코드만 보고 만들었으며, 키 값은 어디에도 적혀 있지 않습니다.

1. 폴더와 파일 한눈에 보기

```
app/inputdata/                      ← 서버의 app 폴더 안에 이 폴더를 통째로 덮어씀 (23개 파일, 약 7,000줄)
├─ dedupe.py                        같은 문제 알아보기 (내용 해시)
├─ schemas.py                       문제 입력 검사 (5가지 유형)
├─ crud/inputdata.py                문제를 DB 에 저장
├─ routers/inputdata.py             기존 일괄 업로드 API
└─ intake/                          ★ 문제지 업로드 기능 전체
   ├─ router.py                     API 주소 (/api/intake/…)
   ├─ service.py                    처리·승인·등록의 중심
   ├─ models.py                     DB 표 4개
   ├─ config.py                     환경변수 스위치
   ├─ llm.py                        AI 호출, .env 읽기
   ├─ parsing.py                    파일 → 문제 레코드, 개인정보 제거
   ├─ prompt.md                     AI 지시문
   ├─ readers/                      파일 읽기·캡처
   │   ├─ pdf_intake.py  xlsx_intake.py  hwp_text.py  ppt_legacy.py
   │   ├─ images.py                 그림·압축 안전
   │   └─ capture.py                원본 캡처
   └─ checks/                       검사·보정
       ├─ checker.py                검사
       ├─ fix.py                    AI 결과 고치기
       └─ suggest.py                조건 예시
```

2. 전체가 어떻게 연결되나

요청 흐름 (실제 함수 이름)

① 올리기 router.create_job → parsing.build_records(readers/*가 파일을 읽고 문제로 나눔) → service.create_job(DB 에 작업·항목 저장) → 202 응답
② 처리 (백그라운드) service.process_job → _process → _run_batches / _handle(문제마다 20개씩 동시) → llm.call_llm → fix.* 로 보정 → _store_rows → classify(checker 검사로 상태 결정)
② 캡처 (백그라운드, 처리와 별개) service.capture_job → readers.capture(문서 → 쪽 이미지) → intake_capture 표에 저장
③ 수정 router.edit_item → service.edit_payload → schemas 검사 → classify 로 다시 검사 → 상태 갱신
④ 승인·등록 router.approve → service.approve → _apply_register → allocate_title(번호 부여) → register_payload → crud.create_problem → dedupe 중복 검사 → problem 표에 저장

파일 사이의 호출 방향 (코드의 import 를 분석한 것)

router → service → llm · parsing → readers/* · checks/* · crud → dedupe

파일	부르는 파일
router.py	service, parsing, models, config
service.py	llm, parsing, checks/checker, checks/fix, checks/suggest, readers/capture, readers/images, crud/inputdata, dedupe, schemas, models, config
parsing.py	readers/pdf_intake, readers/xlsx_intake, readers/images, config

readers/pdf_intake.py	readers/hwp_text, readers/ppt_legacy, readers/images
readers/xlsx_intake.py · hwp_text.py	readers/images
readers/capture.py	readers/pdf_intake
checks/checker.py	checks/fix, config
checks/fix.py	readers/pdf_intake
crud/inputdata.py	dedupe, 서버의 app.problem.models
routers/inputdata.py	crud/inputdata, schemas
models.py · router.py	서버의 app.database (DB), app.security.security (로그인)

> **서버(app)와 만나는 곳은 4군데뿐입니다**: `app.database` (DB 연결·Base), `app.security.security` (로그인 확인), `app.problem.models` (문제 표), `app/main.py` · `app/database.py` (라우터 연결과 표 생성에 각각 한두 줄). 나머지는 이 폴더 안에서 끝납니다.

3. 파일별 기능과 구현 방식

파일	줄	하는 일	어떻게 구현했나	연결
`dedupe.py`	60	같은 문제인지 알아보는 내용 해시. 중복·제목 번호 겹침 오류 정의	유형+제목+설명(공백·대소문자 정리, JSON으로 묶인 설명은 본문만)으로 sha256 을 만든다	부르는 곳: crud/inputdata.py, service.py
`schemas.py`	368	문제 입력 검사 (5가지 유형별)	pydantic 모델. 난이도·채점 방식 정규화, 정답 코드가 비면 거부, `image_url` 검사. 유형별 모델을 묶은 `problem_adapter` 가 입구	부르는 곳: routers/inputdata.py, service.py(수정·등록 검사)
`crud/inputdata.py`	199	문제를 DB 에 저장	유형별로 저장 가능한 칸만 골라 담고, 내용 해시로 중복을 검사한 뒤 flush. 겹침 오류를 중복/번호 겹침으로 구분해 던진다	부르는 곳: routers/inputdata.py, service.py(등록) → 부르는 곳: dedupe.py, 서버 `app.problem.models`
`routers/inputdata.py`	165	기존 일괄 업로드 API (JSON 으로 문제 목록을 받음)	문제마다 `create_problem` 을 부르고 성공·실패를 모아 응답. 저장(commit)은 이 라우터가 한다	main.py 가 `/api/inputdata` 로 연결
`intake/__init__.py`	4	패키지 설명	—	—
`intake/router.py`	216	문제지 업로드 API 11개 (올리기, 진행, 목록, 수정, 승인, 거절, 원본, 조건 예시, 이어하기 등)	FastAPI 라우터. 파일 검사 후 `parsing` 으로 문제 레코드를 만들고 `service` 에 넘긴다. 본인 작업만 접근	main.py 가 `/api/intake` 로 연결 → 부르는 곳: service, parsing, models, config
`intake/service.py`	958	처리·승인·등록의 중심: 작업 관리, AI 처리, 상태 결정, 제목 번호, 등록, 승인 화면 기능	구역 5개(목차 참고). AI 처리는 문제마다 20개씩 동시에 호출하고, 결과를 `fix` 로 고친 뒤 `classify` 로 상태를 정한다	부르는 곳: router.py → 부르는 곳: llm, parsing, checks/*, readers/capture, readers/images, crud, models, config
`intake/models.py`	140	DB 표 4개와 설명 JSON 묶기/풀기	`intake_job`, `intake_item`, `intake_capture`, `problem_title_seq`. 설명은 `{intent, subtitle, content}` 로 묶어 저장	부르는 곳: router, service, 서버 database.py(표 생성)
`intake/config.py`	179	환경변수 스위치(한도·동시성·캡처 등)를 읽는 함수 모음	함수마다 `os.environ.get(이름, 기본값)`. 코드에 기본값이 있어 설정하지 않아도 동작	부르는 곳: 거의 모든 intake 파일

`intake/llm.py`	539	AI 호출 전부: .env 읽기, 제공자 판별, 재시도, 스트리밍, 출력 읽기	키 모양으로 제공자(OpenAI·Anthropic·Gemini·호환) 판별 → 요청 → 혼잡 오류는 3·8·20·45초 간격으로 재시도. 첫 글자부터 받는 스트리밍으로 앞단의 100초 제한을 피한다	부르는 곳: service.py(`_default_ai`)
`intake/parsing.py`	578	파일 → 문제 레코드. 개인정보 가리기	엑셀과 문서를 읽고(readers), 문제 경계를 찾아 한 문제 = 레코드 하나로. 전화번호·학번 등은 AI 에 보내기 전에 지운다. 풀이 파일은 문제 파일과 짝지어 정답 자료로 붙인다	부르는 곳: router.py, service.py → 부르는 곳: config, readers/*
`intake/prompt.md`	169	AI 에게 주는 지시문 (서버 코드 아님)	항목 매핑, 설명 정리 규칙, 유형 판단, 보류 조건. 예시를 넣지 않아 짧게 유지	읽는 곳: service.py (경로는 `INTAKE_PROMPT_PATH` 로 바꿀 수 있음)
`intake/checks/fix.py`	1395	AI 결과를 서버 규칙으로 고침 (이름은 fix 지만 정리·보정·판정 모음)	함수 75개를 구역 9개로 모았다: AI 출력 바로잡기 / 입력 빈칸 / 설명 정리 / 조건·제공 코드 / 정답 / 엑셀 / 제목 / 짧은 AI 호출 / 보류 판정	부르는 곳: service.py, checker.py → 부르는 곳: readers/pdf_intake
`intake/checks/checker.py`	373	AI 결과 검사	스키마, 조건 형식(`[필수|금지|허용|출력형식]`), 정답 코드와 제공 코드 관계, 중복, 설명이 원본과 같은지 확인. 정답 코드 실행 검증은 기본 꺼짐(격리 확인이 있어야 켜짐)	부르는 곳: service.py(`classify`) → 부르는 곳: fix, config
`intake/checks/suggest.py`	103	조건 예시 하나 만들기 (AI 없음)	정답 코드에서 글자 값·주석을 지우고 쓰는 것(라이브러리 함수, 상속, 재귀, 반복문 …)을 규칙으로 읽어 `[필수] … 를 사용한다` 한 줄을 만든다. 이미 같은 조건이 있으면 건너뜀	부르는 곳: service.py(`item_out` , `suggest_conditions`)
`intake/readers/pdf_intake.py`	613	워드·PPT·PDF 글 읽기와 문제 나누기	python-docx(번호 매김·표·그림 표시), pptx, pdftotext. 문제 ID 로 나누고, 제목+번호, 번호만, 문서 전체 순으로 대체. 잡음 정리, 소문항 분리, 유형 힌트, 스캔 PDF 판별과 쪽 그리기	부르는 곳: parsing, checks/fix, readers/capture → 부르는 곳: hwp_text, ppt_legacy, images
`intake/readers/xlsx_intake.py`	292	엑셀·CSV·SQLite 읽기	머리글(이름·설명 …)을 동의어 표로 찾는다. 머리글이 없으면 모든 칸을 글로 읽어 문서처럼 처리	부르는 곳: parsing → 부르는 곳: images
`intake/readers/hwp_text.py`	321	한글 파일 글 읽기	hwp: olefile 로 열어 압축된 본문 스트림을 풀고 글 레코드를 읽음. hwpx: zip 안의 XML	부르는 곳: pdf_intake → 부르는 곳: images
`intake/readers/ppt_legacy.py`	119	옛 `.ppt` 글 읽기	이진 파일의 글 레코드를 직접 읽음 (olefile)	부르는 곳: pdf_intake
`intake/readers/images.py`	148	그림·압축 안전 처리	압축 폭탄 검사(풀린 크기 상한), 그림 종류 확인, 크기 줄이기, 작업당 그림 용량 한도, 글 파일 인코딩 판별	부르는 곳: parsing, service, pdf_intake, xlsx_intake, hwp_text
`intake/readers/capture.py`	144	원본 캡처: 문서 → 쪽 이미지 → 문제와 짝짓기	LibreOffice 로 PDF 로 바꾸고(없으면 건너뜀) 쪽마다 글을 읽어, 문제 원문의 첫·끝 줄이 나오는 쪽을 찾아 JPEG 로 그린다	부르는 곳: service.py(`capture_job`) → 부르는 곳: pdf_intake

service.py 구역 (함수 위치 안내)

구역	대표 함수	역할

1. 작업 만들기·진행 관리	create_job, daily_usage, active_job, job_summary, reset_failed	작업·항목 저장, 하루 한도, 진행 요약, 실패한 문제 이어서 처리
2. AI 처리 흐름	process_job, _process, _run_batches, _handle, _store_rows, _timed	문제마다 AI 호출 → 보정 → 저장. 40초 넘는 호출은 다시 보내고 또 늦으면 보류
3. 상태 결정	classify	검사 결과로 blocked / needs_input / held / needs_review 결정
4. 등록·승인	approve, _apply_register, allocate_title, register_payload	제목 번호 부여, 설명 JSON 묶기, 중복 검사, 저장
5. 승인 화면 기능	edit_payload, original_of, suggest_conditions, capture_job, item_out	수정 후 재검사, 원본 보기, 조건 예시, 원본 캡처

4. DB 표 4개 (서버가 시작할 때 자동으로 만듦)

표	담는 것	비고
`intake_job`	업로드 작업 하나 (상태, 파일 이름, 옵션, 진행 수)	사용자별 조회
`intake_item`	문제 하나의 입력(record)·AI 결과(payload)·검사 기록(review)·상태	작업이 지워지면 함께 삭제
`intake_capture`	원본 캡처 쪽 이미지(JPEG)	30일 뒤 자동 삭제, 개인정보가 이미지에 남을 수 있음
`problem_title_seq`	제목 코드 접두(PY41)별 마지막 번호	한 줄을 원자적으로 +1 → 번호가 겹치지 않음

5. `.env` · 환경변수를 쓰는 곳

.env 를 읽는 코드는 한 곳입니다: `intake/llm.py` 의 `load_dotenv()`

- **언제 읽나**: AI 를 처음 부를 때(`llm.resolve()` → `service._default_ai`). 업로드 요청마다가 아니라 **AI 호출 준비 단계**에서만 읽는다.
- **어디를 찾나** (앞에서부터, 있는 파일을 모두): ① 서버를 실행한 현재 폴더 ② `intake/` 폴더 ③ `inputdata/` ④ `app/` 폴더. 보통 서버가 이미 쓰는 `app/.env` 가 ④번에서 잡힌다.
- **우선순위**: **이미 설정된 환경변수가 이긴다.** `.env` 는 아직 없는 변수, 그리고 값이 비어 있지 않은 것만 채운다 (Docker 의 환경변수를 `.env` 가 덮어쓰지 않는다).
- **읽는 규칙** (`parse_dotenv`): `이름=값` 한 줄씩, `#` 주석, `export` 접두 허용, 따옴표 제거, 값 뒤 `# 주석` 제거, 이름은 영문·숫자·밑줄만.
- **서버 기존 코드(app)는 따로 읽는다**: `app/database.py` 가 python-dotenv 로 자기 `.env` (DB 주소, 비밀키 등)를 읽는다. 두 곳이 같은 파일을 읽어도 서로 충돌하지 않는다.

> **필요한 최소 설정은 AI 키 하나뿐입니다.** `GEMINI_API_KEY` · `ANTHROPIC_API_KEY` · `OPENAI_API_KEY` · `LLM_API_KEY` 중 아무거나 서버의 환경변수 또는 `app/.env` 에 한 줄 넣으면 됩니다 (키의 앞부분으로 제공자를 자동 판별). 나머지는 모두 코드에 기본값이 있어 적지 않아도 동작합니다.

5-1. AI 설정 (llm.py)

변수	읽는 곳	기본값	뜻
`GEMINI_API_KEY` · `GOOGLE_API_KEY`	llm.py 18행 (`KEY_VARS`)	—	구글(Gemini) 키
`ANTHROPIC_API_KEY`	llm.py 18행	—	Claude 키
`OPENAI_API_KEY` · `LLM_API_KEY`	llm.py 18행	—	OpenAI 또는 OpenAI 호환 서비스 키
`LLM_PROVIDER`	llm.py 63행	키 모양으로 자동 판별	`google` / `anthropic` / `openai` . 키가 여러 개일 때만 지정
`LLM_MODEL`	llm.py 364행	제공자별 기본 모델	모델 이름. OpenAI 호환 서비스는 기본값이 없어 필수
`LLM_BASE_URL`	llm.py 272행	OpenAI 주소	OpenAI 호환 서비스 주소

`LLM_THINKING`	llm.py 299행, service.py 222행	모델 기본값	생각 정도
`LLM_STREAM` · `LLM_STREAM_USAGE`	llm.py 273·277행	스트리밍 on	끄면 한 번에 받음. 앞단 게이트웨이의 100초 제한 때문에 기본 on
`LLM_CACHE_CONTROL`	llm.py 264행	off	Anthropic 프롬프트 캐시 표시
`LLM_MAX_TOKENS_PARAM` · `LLM_TEMPERATURE` · `LLM_EXTRA_BODY`	llm.py 266~270행	—	서비스마다 다른 요청 값을 코드 수정 없이 지정
`LLM_DOTENV_CWD_ONLY`	llm.py 39행	—	`1` 이면 현재 폴더의 .env 만 읽음 (시험용: 실제 서버 .env 가 섞이지 않게)

5-2. 업로드 기능 스위치 (config.py 외)

변수	기본값	뜻
`INTAKE_MAX_FILES` · `INTAKE_MAX_FILE_MB`	`5` · `20`	한 번에 올릴 파일 수 · 한 파일 최대 MB
`INTAKE_MAX_PROBLEMS`	`200`	작업 하나의 최대 문제 수
`INTAKE_DAILY_JOBS` · `INTAKE_DAILY_PROBLEMS`	`30` · `500`	사용자 한 명의 하루 작업 수 · 문제 수 한도
`INTAKE_CONCURRENCY`	`20`	AI 를 동시에 부르는 문제 수 (서비스의 동시 호출 한도에 걸리면 낮춤)
`INTAKE_CONCURRENT_JOBS`	`2`	동시에 처리하는 작업 수
`INTAKE_CALL_DEADLINE` · `INTAKE_CALL_HARD_LIMIT`	`40` · `40`	AI 호출이 이 초를 넘으면 같은 요청을 다시, 또 늦으면 그 문제만 보류
`INTAKE_AI_ITEM_TIMEOUT` · `INTAKE_HEARTBEAT_SEC`	`900` · `120`	문제 하나 처리 상한 · 처리 중 표시 갱신 주기(멈춘 작업 판별)
`INTAKE_MAX_CONSEC_FAIL`	`3`	연속 AI 실패가 이만큼이면 작업을 멈춤(나중에 이어하기)
`INTAKE_MAX_TOKENS`	`20000`	AI 출력 최대 토큰
`INTAKE_REPAIR_ROUNDS`	`1`	서버 검사에서 막힌 결과를 AI 가 다시 쓰는 횟수 (0 이면 끔)
`INTAKE_FILL_META` · `INTAKE_FILL_CODE`	`on` · `on`	부제목·문제의도 / 비어 있는 정답 코드를 짧은 AI 호출로 채움
`INTAKE_LIGHT_ROWS` · `INTAKE_WARMUP` · `INTAKE_BATCH`	`on` · `on` · `1`	완성된 엑셀 행 빠른 처리 / 프롬프트 캐시 데우기 / 짧은 문제를 N개씩 한 번에 묶어 호출 (1=끔, 최대 8. 앞 문제 내용이 섞일 수 있어 실제 AI 로 비교한 뒤 켬)
`INTAKE_AI_ANSWER_REVIEW`	`on`	AI 가 직접 푼 객관식·단답형 정답은 승인 사유로 표시
`INTAKE_REDACT_PII`	`on`	AI 로 보내기 전에 전화번호·학번 등 제거
`INTAKE_WHOLE_DOC` · `INTAKE_WHOLE_DOC_CHARS`	`off` · `24000`	문제 경계를 못 찾으면 문서 전체를 AI 에 보내 나누게 함
`INTAKE_SCAN_PAGES` · `INTAKE_SCAN_CHUNK`	`40` · `4`	스캔 PDF 최대 쪽 수 · 한 번에 AI 에 보내는 쪽 수
`INTAKE_CAPTURE` · `_PAGES` · `_DPI` · `_DAYS`	`on` · `40` · `100` · `30`	원본 캡처 사용 · 파일당 최대 쪽 · 해상도 · 보관 일수
`INTAKE_EXEC` · `INTAKE_EXEC_ISOLATED` · `INTAKE_EXEC_NO_NET`	`off` · `off` · `on`	정답 코드 서버 실행 검증. 앞의 둘을 **모두** 켜야 켜짐(격리 컨테이너 확인용), 켜면 네트워크는 끊고 실행
`INTAKE_MAX_JOB_IMAGE_MB` · `INTAKE_MAX_UNZIPPED_MB`	코드 기본값 · `300`	작업당 그림 용량 · 압축을 풀었을 때 최대 크기(압축 폭탄 방지)
`INTAKE_PROMPT_PATH`	`intake/prompt.md`	AI 지시문 파일 위치

`POPPLER_PATH` · `LIBREOFFICE_PATH`	`PATH` 에서 찾음	PDF 도구(pdftotext, pdftoppm) 폴더 · LibreOffice 실행 파일 (캡처용)

5-3. 키·비밀값을 다루는 방식

- **키는 AI 호출의 요청 헤더로만 쓴다.** 코드에서 키를 쓰는 곳은 `llm.resolve()` → `call_llm()` 한 줄기이고, 로그에 찍히는 `key` 는 모두 **문제 이름**이다. 키 값은 응답·DB·로그 어디에도 쓰지 않는다 (안내 문구에도 키 값은 없다).
- **정답 코드를 실행할 때는 서버 환경변수를 넘기지 않는다.** `checker._safe_env()` 가 실행 프로세스에 `PATH` 등 최소한만 준다 (API 키 제외). 실행 검증 자체가 기본 꺼짐이다.
- **배포 파일에 `.env` 는 들어 있지 않다.** `inputdata_full.zip` (23개 파일)에는 소스와 `prompt.md` 만 있다. `.env` 는 서버에서만 관리한다.
- **AI 로 보내기 전**에 문서의 전화번호·이메일·주민번호·`학번:`·`성명:` 줄을 지운다 (`INTAKE_REDACT_PII`). 원본 캡처 이미지에는 개인정보가 남을 수 있어 30일 뒤 지운다.

6. API 11개 (모두 `/api/intake` 아래, 로그인 필요)

메서드 · 경로	연결되는 함수	하는 일
`POST /jobs`	router.create_job → parsing → service.create_job	파일 올리기 (202). 문제로 나눈 뒤 백그라운드 처리·캡처 시작
`POST /preview`	router.preview → parsing	AI 없이 문제 수·경계 미리 확인
`GET /jobs` · `GET /jobs/{id}`	service.job_summary	작업 목록 · 진행과 상태별 개수
`GET /jobs/{id}/items`	service.item_out	항목 목록 (+ 부제목·의도·조건 예시)
`GET /items/{id}/original`	service.original_of	문제 하나의 원본(글·엑셀 열·그림·캡처)
`PATCH /items/{id}`	service.edit_payload → classify	수정 저장 + 서버 재검사
`POST /items/{id}/suggest-conditions`	service.suggest_conditions → suggest	조건 예시 하나
`POST /jobs/{id}/approve`	service.approve → _apply_register	승인(등록)
`POST /items/{id}/reject`	router.reject_item	거절
`POST /jobs/{id}/resume`	service.reset_failed → process_job	AI 오류로 못 한 문제 이어서

7. 알아 둘 점

- 실제 PDF 캡처·실제 Postgres·서버 환경의 패키지(`python-docx` , `openpyxl` , `olefile` , `Pillow` , LibreOffice, poppler)는 개발 PC에서 확인하지 못했다. 서버에 반영한 뒤 파일 하나로 시험해 보는 것을 권한다.
- `fix.py` (1,395줄)와 `service.py` (958줄)는 구역 목차를 붙였지만 한 파일에 규칙이 많다. 더 쪼개야 하면 구역 단위로 나누면 된다.
- AI 판단은 실행마다 조금씩 달라서 같은 문서도 보류·승인 대기 구분이 다르게 나올 수 있다.